RETAIL SALES & CUSTOMER PERFORMANCE ANALYSIS

PROJECT REPORT

Submitted by: Rohit Rajendra Gaikwad

Course / Programme:Data Analytics & Data Science Intern

Internship / Training Organization: StarCentauri Technologies Pvt. Ltd.

CONTENTS

1. Cover Page
2. Executive Summary
3. Introduction
4. Business Problem
5. Objectives
6. Dataset & Data Dictionary
7. Tools & Technologies
8. Data Cleaning
9. Data Transformation
10. Exploratory Data Analysis
11. SQL Analysis
12. KPI Development
13. Power BI Dashboard
14. Insights
15. Recommendations
16. Challenges
17. Conclusion
18. Future Scope
19. References
20. Appendix A — Python Workflow
21. Appendix B — SQL Query Library

2. EXECUTIVE SUMMARY

This project analyses the Global Superstore retail dataset to understand sales, profitability, customers, products, and regional performance. The workflow covers data understanding, cleaning, transformation, exploratory analysis in Python, SQL-based analysis, KPI definition, and a three-page Power BI dashboard.

The transformed dataset contains 51,290 transaction rows and 35 columns, covering orders dated 01 January 2011 to 31 December 2014. It records $12,642,501.91 in sales and $1,467,457.29 in profit, with 25,035 distinct orders, 1,590 distinct customers, 10,292 distinct product IDs, and 178,312 units sold. The overall profit margin is 11.61%, and average order value is $504.99.

The Power BI report is separated into three pages: Executive Overview, Customer Insights, and Product Performance. This division keeps the overview uncluttered while providing dedicated customer and product analysis. All values in this report were calculated from the supplied transformed CSV; Power BI cards may display rounded values.

3. INTRODUCTION

Retail businesses generate large amounts of transaction data. Analysing this data can reveal which products and categories drive revenue, how profitability varies across regions, which customers contribute the most value, and how sales change over time.

This project applies a practical analytics lifecycle to a Global Superstore dataset. Python is used for inspection, cleaning, transformation, and visual exploration; MySQL is used to query the imported transaction table; and Power BI is used to communicate the results through interactive dashboards.

4. BUSINESS PROBLEM

The raw transaction data contains many rows and attributes. Without structured analysis, it is difficult for decision-makers to compare performance across time, categories, regions, customers, and products. Management needs a consistent way to monitor revenue and profit, identify high-value customers and products, find weak-performing areas, and explore results using filters.

The project addresses this problem by creating a cleaned and transformed dataset, a set of reusable SQL queries and KPI definitions, and a Power BI report that supports exploratory decision-making.

5. OBJECTIVES

- Inspect the dataset structure, data types, missing values, duplicates, and date coverage.
- Clean and standardise fields needed for analysis while preserving transaction-level records.
- Create date features and business categories to support analysis.
- Use Python to calculate descriptive statistics and visualise sales and profit patterns.
- Use SQL to answer business questions involving totals, grouping, ranking, and monthly growth.
- Define management KPIs with clear formulas and interpretations.
- Build an interactive three-page Power BI dashboard for executive, customer, and product analysis.
- Translate analytical results into practical recommendations and identify limitations.

6. DATASET AND DATA DICTIONARY

Source file: transformed_retail_sales(1).csv (the transformed copy supplied with this report). A cleaned file, Cleaned_superstore_dataset2011-2015.csv, was also supplied. The transformed file contains 51,290 rows and 35 columns. The cleaned file contains 51,290 rows and 23 columns. The source records cover 2011–2014 according to Order Date; the filename should not be interpreted as proof that 2015 transactions are present.

The cleaned dataset has 23 columns after dropping Postal Code. The transformed dataset has 35 columns after adding derived fields. Postal Code was removed during cleaning because it had extensive missingness and was not required for the planned analysis.

Row ID	Transaction row identifier	Row-level traceability
Order ID, Order Date, Ship Date	Order identifier and transaction/shipping dates	Order counts and time analysis
Customer ID, Customer Name, Segment	Customer attributes	Customer-level analysis
Product ID, Product Name, Category, Sub-Category	Product identifiers and hierarchy	Product and category analysis
Sales, Profit, Quantity, Discount, Shipping Cost	Numeric transaction measures	Revenue, profit, units and discount analysis
Country, Market, Region, State, City	Geographic attributes	Regional comparisons
Ship Mode, Order Priority	Fulfilment and priority attributes	Operational segmentation
Year, Month, Month_Name, Quarter	Derived date features	Time-series analysis
Profit_Margin, Order_Value	Derived numeric fields	Margin and order-value analysis
Customer_Segment, Sales_Category, Profit_Category	Derived classification fields	Business segmentation
Customer_Type, Product_Performance, Order_Size	Derived classification fields	Repeat customer and performance analysis

7. TOOLS AND TECHNOLOGIES

Python	Data loading, validation, cleaning, transformation, descriptive statistics, and visualisation
Pandas	Tabular data processing and group-by calculations
NumPy	Numeric operations where needed
Matplotlib / Seaborn	Charts for exploratory data analysis
Jupyter Notebook	Interactive, reproducible Python workflow
MySQL	Relational storage and SQL analysis
MySQL Workbench	Database management and query execution
Power BI Desktop	Data modelling, DAX measures, interactive report pages
DAX	KPI and ranking calculations
CSV	Data exchange between Python, MySQL, and Power BI

8. DATA CLEANING

The cleaning phase prepared the source data for reliable analysis. The cleaned output supplied for review contains 51,290 rows and 23 columns.

1. Loaded the source CSV into pandas and inspected shape, column names, data types, and sample records.
2. Checked duplicate rows. The supplied transformed dataset has zero fully duplicated rows according to pandas duplicated().
3. Inspected missing values. Postal Code was removed because it had extensive missingness and was not essential to the selected business questions.
4. Converted Order Date and Ship Date to date/datetime values.
5. Checked numeric fields such as Sales, Quantity, Discount, Profit, and Shipping Cost for appropriate numeric types.
6. Saved the cleaned dataset as Cleaned_superstore_dataset2011-2015.csv.

Quality-control result: the cleaned file has 51,290 rows and 23 columns. The transformed file has the same number of rows, so the transformation stage added derived columns rather than aggregating away transaction records.

9. DATA TRANSFORMATION

The transformation phase added fields that simplify analysis and report building. The supplied transformed CSV has 35 columns.

Year	Year extracted from Order Date
Month	Numeric month number, used for chronological sorting
Month_Name	Readable month label
Quarter	Quarter extracted from the order date
Profit_Margin	Derived margin field; use the total-level KPI for overall margin
Order_Value	Set to Sales for this workflow; not a unit price
Customer_Segment	High Value, Medium Value, or Low Value classification
Sales_Category	Sales-level classification
Profit_Category	Profitability classification
Customer_Type	Customer type such as repeat customer, based on the implemented rule
Product_Performance	Product performance classification
Order_Size	Order size classification

The transformation output was saved as transformed_retail_sales.csv. Date-derived fields support month, quarter, and year reporting. Business labels support segmented analysis; thresholds should be documented in the notebook so that the classification can be reproduced.

10. EXPLORATORY DATA ANALYSIS

The transformed dataset contains 51,290 rows, 1,590 distinct customers, 25,035 distinct orders, and 10,292 distinct product IDs. Total sales are $12,642,501.91, and total profit is $1,467,457.29.

The EDA tasks covered total sales, monthly sales, category sales, regional sales, top and bottom products, and profit analysis. Visualisations included line charts for monthly trends, bar charts for category and region comparisons, product ranking charts, and distributions of numeric measures.

Furniture	$4,110,874.19	$285,204.72	6.94%
Office Supplies	$3,787,070.23	$518,473.83	13.69%
Technology	$4,744,557.50	$663,778.73	13.99%

Yearly sales:

2011	$2,259,450.90	N/A
2012	$2,677,438.69	18.50%
2013	$3,405,746.45	27.20%
2014	$4,299,865.87	26.25%

Regional sales ranking (selected leading regions):

Central	$2,822,302.52	$311,403.98
South	$1,600,907.04	$140,355.77
North	$1,248,165.60	$194,597.95
Oceania	$1,100,184.61	$120,089.11
Southeast Asia	$884,423.17	$17,852.33
North Asia	$848,309.78	$165,578.42

Product examples by revenue:

Apple Smart Phone, Full Size	$86,935.78
Cisco Smart Phone, Full Size	$76,441.53
Motorola Smart Phone, Full Size	$73,156.30
Nokia Smart Phone, Full Size	$71,904.56
Canon imageCLASS 2200 Advanced Copier	$61,599.82

Interpretation: Technology has the highest category sales and profit in the supplied data. Office Supplies has the highest distinct order count among the three categories and a higher profit margin than Furniture. Furniture has substantial sales but a lower margin. Annual sales increased each year from 2011 through 2014 in the dataset. These are descriptive findings, not proof of the causes behind the changes.

11. SQL ANALYSIS

The transformed data was imported into MySQL database retail_analysis, table retail_sales_import. The SQL tasks were designed to practise SELECT, aggregate functions, WHERE, GROUP BY, ORDER BY, HAVING, CASE, joins, common table expressions, and window functions. Because source column names contain spaces, use backticks around names such as `Order ID` and `Customer ID`.

The following examples represent the core SQL analysis questions. The complete query library is included in Appendix B.

1. Total sales

```
SELECT SUM(Sales) AS total_sales FROM retail_sales_import;
```

2. Total profit

```
SELECT SUM(Profit) AS total_profit FROM retail_sales_import;
```

3. Total orders

```
SELECT COUNT(DISTINCT `Order ID`) AS total_orders FROM retail_sales_import;
```

4. Total customers

```
SELECT COUNT(DISTINCT `Customer ID`) AS total_customers FROM retail_sales_import;
```

5. Average order value

```
SELECT SUM(Sales) / COUNT(DISTINCT `Order ID`) AS average_order_value FROM retail_sales_import;
```

6. Sales by category

```
SELECT Category, SUM(Sales) AS total_sales FROM retail_sales_import GROUP BY Category ORDER BY total_sales DESC;
```

7. Sales by region

```
SELECT Region, SUM(Sales) AS total_sales FROM retail_sales_import GROUP BY Region ORDER BY total_sales DESC;
```

8. Profit by category

```
SELECT Category, SUM(Profit) AS total_profit FROM retail_sales_import GROUP BY Category ORDER BY total_profit DESC;
```

9. Profit by region

```
SELECT Region, SUM(Profit) AS total_profit FROM retail_sales_import GROUP BY Region ORDER BY total_profit DESC;
```

10. Monthly sales

```
SELECT Year, Month, SUM(Sales) AS monthly_sales FROM retail_sales_import GROUP BY Year, Month ORDER BY Year, Month;
```

12. KPI DEVELOPMENT

KPIs provide consistent definitions for monitoring performance. Measures were created in DAX for the Power BI report and can be compared with equivalent SQL aggregates.

Total Revenue	SUM(Sales)	Total value of sales recorded
Total Profit	SUM(Profit)	Net profit recorded in the transaction data
Total Orders	Distinct count of Order ID	Number of unique orders
Total Customers	Distinct count of Customer ID	Number of unique customers
Average Order Value (AOV)	Total Revenue ÷ Total Orders	Average sales value per distinct order
Profit Margin %	Total Profit ÷ Total Revenue × 100	Profit generated per unit of sales
Sales Growth %	(Current-period sales − previous-period sales) ÷ previous-period sales × 100	Change in sales compared with prior period
Products Sold	SUM(Quantity)	Total units sold

Calculated overall KPIs: revenue $12,642,501.91; profit $1,467,457.29; orders 25,035; customers 1,590; AOV $504.99; profit margin 11.61%; units sold 178,312. These calculations use the full transformed dataset without report-page filters.

Important distinction: average customer spending equals total revenue divided by distinct customers; average order value equals total revenue divided by distinct orders. They are different measures and should not be labelled interchangeably.

13. POWER BI DASHBOARD

The Power BI report was designed as three pages to separate executive monitoring from detailed customer and product analysis. The pages use a consistent light background, rounded white cards, navy headings, and blue/orange/teal accents.

Page 1 — Executive Overview	Management-level summary	**Six KPI cards:** Total Revenue, Total Profit, Total Orders, Total Customers, AOV, Profit Margin. **Charts:** revenue by month, profit by month, revenue by category, revenue by region, and top products by revenue. **Slicers:** Year, Month, Region, Category.
Page 2 — Customer Insights	Customer value and behaviour	**Six KPI cards:** Total Customers, Customer Revenue, Total Orders, Average Customer Spending, Repeat Customers, Top Customer Revenue. **Charts:** Top 10 customers by revenue, customer segment distribution, customer ranking/spending table, revenue by customer segment.
Page 3 — Product Performance	Product and category results	**Six KPI cards:** Product Revenue, Product Profit, Total Products, Products Sold, Product Orders, Product Profit Margin. **Charts:** Top 10 products by revenue, bottom 10 products by revenue, product ranking table, revenue vs profit by category.

Interactivity includes Year, Month, Region, Category, and customer-segment slicers where relevant; cross-filtering; category-to-subcategory drill-down in the product matrix; and tooltips for additional product measures. Sidebar buttons provide page navigation. In Power BI Desktop edit mode, Ctrl+Click is normally needed to activate buttons; in Reading view, buttons can be used with a normal click.

Model design: FactSales stores transaction-level records. **DimCustomer, DimProduct,** and **DimDate** provide descriptive attributes for filtering and grouping. Dimension keys should be unique, relationships should be active one-to-many relationships from dimensions to FactSales, and the date relationship should connect date-only values.

14. INSIGHTS

- Overall sales were $12,642,501.91, with $1,467,457.29 profit and an overall margin of 11.61%.
- Technology contributed the largest category revenue ($4,744,557.50) and profit ($663,778.73).
- Office Supplies achieved a 13.69% category profit margin, compared with 6.94% for Furniture.
- Central was the largest region by sales in the supplied dataset ($2,822,302.52).
- Sales increased from $2,259,450.90 in 2011 to $4,299,865.87 in 2014; the calculated year-over-year growth was 18.50% in 2012, 27.20% in 2013, and 26.25% in 2014.
- The transformed customer segmentation contains 838 High Value, 271 Medium Value, and 481 Low Value customers. The segment labels reflect the transformation rules used in the project and should not be treated as independently validated customer-lifetime-value tiers.
- The top-revenue product in the supplied data was Apple Smart Phone, Full Size with $86,935.78 in sales.
- 1,580 customers placed more than one distinct order in the complete dataset under the repeat-customer definition used here.

These findings describe the dataset and do not establish causality. Business explanations should be validated with additional context such as costs, pricing, campaigns, inventory, and market conditions.

15. RECOMMENDATIONS

- Review product-level profitability, not revenue alone. A high-revenue product may still have a low margin or negative profit.
- Investigate Furniture profitability and subcategory performance to identify products or discounts that reduce margin.
- Use the regional dashboard to compare both revenue and profit before allocating sales effort; high sales do not automatically mean high profitability.
- Retain high-value customers through suitable service and retention strategies, while testing targeted offers for medium-value customers.
- Validate the customer-segmentation thresholds and document them so that business users understand how High, Medium, and Low Value labels are assigned.
- Monitor monthly revenue and profit together. Investigate unusual declines or margin changes rather than reacting to sales volume alone.
- Refresh the dataset on a defined schedule and check row counts, date coverage, duplicate IDs, missing values, and measure reconciliation after each refresh.

16. CHALLENEGES

- Handling a large transaction file with over fifty thousand rows and many categorical fields.
- Removing the highly incomplete Postal Code field while retaining the remaining transaction records.
- Creating consistent date and business-derived fields for analysis.
- Importing the CSV into MySQL and ensuring that the imported schema and data types matched the source.
- Working with column names containing spaces in SQL queries.
- Building a star-schema model in Power BI and ensuring dimension keys were unique before creating one-to-many relationships.
- Troubleshooting customer-ranking measures and checking whether DAX measures responded correctly to customer context.
- Balancing six KPI cards and multiple charts across dashboard pages while maintaining readable labels and interactive navigation.

17. CONCLUSION

The project demonstrates an end-to-end retail analytics workflow using Python, SQL, and Power BI. The data was inspected, cleaned, transformed, analysed, and used to define KPIs and build three interactive report pages. The resulting report provides a structured view of sales, profit, customer behaviour, product performance, and regional results.

The project also highlights the importance of validating metric definitions, data relationships, and ranking logic. The dashboard is a decision-support tool; further investigation is required before using descriptive patterns to make causal claims or major operational decisions.

18. FUTURE SCOPE

- Automate data refresh and validation checks.
- Add forecasting for monthly revenue and profit.
- Create a dedicated low-margin and loss-making product analysis.
- Add customer cohort and retention analysis.
- Integrate inventory and stock-out information to connect sales with availability.
- Publish the report to Power BI Service with role-based access where licensing permits.
- Add drill-through pages for individual customers and products.
- Create automated alerts for low profit margin, declining sales, and unusual regional changes.

19. REFERENCES

- Global Superstore dataset supplied for the project: superstore_dataset2011-2015.csv, cleaned and transformed copies.
- Python pandas documentation: https://pandas.pydata.org/docs/
- Seaborn documentation: https://seaborn.pydata.org/
- MySQL Reference Manual: https://dev.mysql.com/doc/
- Microsoft Power BI documentation: https://learn.microsoft.com/power-bi/
- DAX function reference: https://learn.microsoft.com/dax/

20. APPENDIX A — PYTHON WORKFLOW

The code below is a compact reproducible workflow using the transformed CSV. It covers dataset loading, checks, summary KPIs, EDA groupings, and output exports. Update the file path to match your computer.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("transformed_retail_sales.csv")
df["Order Date"] = pd.to_datetime(df["Order Date"], errors="coerce")
df["Ship Date"] = pd.to_datetime(df["Ship Date"], errors="coerce")

# 1. Basic data understanding
print("Shape:", df.shape)
print(df.info())
print(df.isnull().sum().sort_values(ascending=False).head(10))
print("Duplicate rows:", df.duplicated().sum())

# 2. Core KPIs
total_sales = df["Sales"].sum()
total_profit = df["Profit"].sum()
total_orders = df["Order ID"].nunique()
total_customers = df["Customer ID"].nunique()
aov = total_sales / total_orders if total_orders else 0
profit_margin = (total_profit / total_sales * 100) if total_sales else 0

print("Total Sales:", round(total_sales, 2))
print("Total Profit:", round(total_profit, 2))
print("Total Orders:", total_orders)
print("Total Customers:", total_customers)
print("AOV:", round(aov, 2))
print("Profit Margin %:", round(profit_margin, 2))
```

```
# 3. Monthly sales, sorted chronologically
monthly_sales = (
   df.groupby(["Year", "Month"], as_index=False)["Sales"]
     .sum()
     .sort_values(["Year", "Month"])
)
sns.lineplot(data=monthly_sales, x="Month", y="Sales", hue="Year", marker="o")
plt.title("Monthly Sales by Year")
plt.tight_layout()
plt.show()

# 4. Category performance
category_summary = df.groupby("Category", as_index=False).agg(
   Sales=("Sales", "sum"),
   Profit=("Profit", "sum"),
   Orders=("Order ID", "nunique")
)
category_summary["Profit Margin %"] = (
   category_summary["Profit"] / category_summary["Sales"] * 100
)
print(category_summary.sort_values("Sales", ascending=False))

sns.barplot(data=category_summary, x="Category", y="Sales")
plt.title("Sales by Category")
plt.tight_layout()
plt.show()

# 5. Regional performance
region_summary = df.groupby("Region", as_index=False).agg(
   Sales=("Sales", "sum"), Profit=("Profit", "sum")
).sort_values("Sales", ascending=False)
print(region_summary)

# 6. Top and bottom products by sales
product_summary = df.groupby("Product Name", as_index=False).agg(
   Sales=("Sales", "sum"), Profit=("Profit", "sum")
)
top10 = product_summary.nlargest(10, "Sales")
bottom10 = product_summary.nsmallest(10, "Sales")
top10.to_csv("Top_10_Products.csv", index=False)
bottom10.to_csv("Bottom_10_Products.csv", index=False)

# 7. Customer analysis
```

```
customer_summary = df.groupby(
    ["Customer ID", "Customer Name"], as_index=False
).agg(
    Revenue=("Sales", "sum"),
    Orders=("Order ID", "nunique")
)
customer_summary["Average Spending per Order"] = (
    customer_summary["Revenue"] / customer_summary["Orders"]
)
customer_summary = customer_summary.sort_values("Revenue", ascending=False)
customer_summary.to_csv("Customer_Analysis.csv", index=False)

# 8. Product performance export
product_summary["Profit Margin"] = (
    product_summary["Profit"] / product_summary["Sales"] * 100
)
product_summary.to_csv("Product_Performance.csv", index=False)

# 9. Monthly and category exports for reporting
monthly_sales.to_csv("Monthly_Sales.csv", index=False)
category_summary.to_csv("Category_Performance.csv", index=False)
region_summary.to_csv("Regional_Performance.csv", index=False)
```

Note: The original cleaning and transformation notebooks should remain the authoritative record of each cleaning rule and classification threshold. This appendix is a compact workflow example, not a substitute for including every cell from the original notebooks if your instructor requires full code evidence.

21. APPENDIX B — SQL QUERY LIBRARY

Run these queries in MySQL Workbench after selecting database retail_analysis. The imported table is retail_sales_import. The table uses spaces in several column names, so keep the backticks.

1. Total sales

```
SELECT SUM(Sales) AS total_sales FROM retail_sales_import;
```

2. Total profit

```
SELECT SUM(Profit) AS total_profit FROM retail_sales_import;
```

3. Total orders

```
SELECT COUNT(DISTINCT `Order ID`) AS total_orders FROM retail_sales_import;
```

4. Total customers

```
SELECT COUNT(DISTINCT `Customer ID`) AS total_customers FROM retail_sales_import;
```

5. Average order value

```
SELECT SUM(Sales) / COUNT(DISTINCT `Order ID`) AS average_order_value FROM retail_sales_import;
```

6. Sales by category

```
SELECT Category, SUM(Sales) AS total_sales FROM retail_sales_import GROUP BY Category ORDER BY total_sales DESC;
```

7. Sales by region

```
SELECT Region, SUM(Sales) AS total_sales FROM retail_sales_import GROUP BY Region ORDER BY total_sales DESC;
```

8. Profit by category

```
SELECT Category, SUM(Profit) AS total_profit FROM retail_sales_import GROUP BY Category ORDER BY total_profit DESC;
```

9. Profit by region

```
SELECT Region, SUM(Profit) AS total_profit FROM retail_sales_import GROUP BY Region ORDER BY total_profit DESC;
```

10. Monthly sales

```
SELECT Year, Month, SUM(Sales) AS monthly_sales FROM retail_sales_import GROUP BY Year, Month ORDER BY Year, Month;
```

11. Top 10 customers

```
SELECT `Customer ID`, `Customer Name`, SUM(Sales) AS revenue FROM retail_sales_import GROUP BY `Customer ID`, `Customer Name` ORDER BY revenue DESC LIMIT 10;
```

12. Repeat customers

```
SELECT `Customer ID`, `Customer Name`, COUNT(DISTINCT `Order ID`) AS order_count FROM retail_sales_import GROUP BY `Customer ID`, `Customer Name` HAVING COUNT(DISTINCT `Order ID`) > 1 ORDER BY order_count DESC;
```

13. Average customer spending

```
SELECT AVG(customer_revenue) AS average_customer_spending FROM (SELECT `Customer ID`, SUM(Sales) AS customer_revenue FROM retail_sales_import GROUP BY `Customer ID`) AS c;
```

14. Top products by sales

```
SELECT `Product ID`, `Product Name`, SUM(Sales) AS revenue FROM retail_sales_import GROUP BY `Product ID`, `Product Name` ORDER BY revenue DESC LIMIT 10;
```

15. Bottom products by sales

```
SELECT `Product ID`, `Product Name`, SUM(Sales) AS revenue FROM retail_sales_import GROUP BY `Product ID`, `Product Name` ORDER BY revenue ASC LIMIT 10;
```

16. Most profitable products

```
SELECT `Product ID`, `Product Name`, SUM(Profit) AS total_profit FROM retail_sales_import GROUP BY `Product ID`, `Product Name` ORDER BY total_profit DESC LIMIT 10;
```

17. Customer ranking (window function)

```
SELECT `Customer ID`, `Customer Name`, SUM(Sales) AS revenue, DENSE_RANK() OVER (ORDER BY SUM(Sales) DESC) AS customer_rank FROM retail_sales_import GROUP BY `Customer ID`, `Customer Name` ORDER BY customer_rank, `Customer Name`;
```

18. Product ranking (window function)

```
SELECT `Product ID`, `Product Name`, SUM(Sales) AS revenue, DENSE_RANK() OVER (ORDER BY SUM(Sales) DESC) AS product_rank FROM retail_sales_import GROUP BY `Product ID`, `Product Name` ORDER BY product_rank, `Product Name`;
```

19. Monthly sales growth (CTE + window function)

```
WITH monthly AS (SELECT Year, Month, SUM(Sales) AS sales FROM retail_sales_import GROUP BY Year, Month), compared AS (SELECT Year, Month, sales, LAG(sales) OVER (ORDER BY Year, Month) AS previous_sales FROM monthly) SELECT Year, Month, sales, previous_sales, ROUND((sales - previous_sales) / NULLIF(previous_sales, 0) * 100, 2) AS growth_pct FROM compared ORDER BY Year, Month;
```

20. Category ranking

```
SELECT Category, SUM(Sales) AS revenue, RANK() OVER (ORDER BY SUM(Sales) DESC) AS category_rank FROM retail_sales_import GROUP BY Category ORDER BY category_rank;
```